할당 프로세스 (Assignment Process)

목차

- 개요
- Halo2 할당 시스템
- assign 메서드 패턴
- CellManager를 통한 셀 할당
- Region 관리
- 회로별 할당 예시
- Challenge 값 사용
- 최적화 기법

개요

할당 프로세스(Assignment Process)는 생성된 Witness 데이터를 실제 회로의 셀(cell)에 할당하는 과정입니다. 이 과정은 Halo2의 `Layouter` API를 사용하여 수행되며, 각 회로는 자신의 제약 조건을 만족하도록 witness 값을 적절한 위치에 배치합니다.

할당 프로세스의 역할

1. **Witness 데이터를 회로 셀에 매핑**: Block에서 추출한 witness 데이터를 회로의 advice 컬럼에 할당
2. **제약 조건 만족**: 회로가 정의한 제약 조건을 만족하도록 값 배치
3. **Challenge 값 적용**: RLC 등에 사용되는 challenge 값을 witness에 적용

4. **효율적인 공간 사용**: CellManager를 통해 셀 공간을 효율적으로 활용

할당 프로세스의 단계

```
Block Witness
|
▼
회로별 데이터 추출
|
▼
Challenge 값 적용
|
▼
Region 생성
|
▼
셀에 값 할당
|
▼
제약 조건 검증
```

Halo2 할당 시스템

Layouter API

Halo2는 `Layouter` 트레이트를 통해 회로 레이아웃을 관리합니다:

```
pub trait Layouter<F: Field> {
fn assign_region<A, AR, N, NR>(
&mut self,
name: N,
assignment: A,
) -> Result<AR, Error>
where
A: FnMut(Region<F>) -> Result<AR, Error>,
N: Fn() -> NR,
NR: Into<String>;
fn assign_table<A, N, NR>(
&mut self,
name: N,
assignment: A,
) -> Result<(), Error>
where
A: FnMut(Table<F>) -> Result<(), Error>,
N: Fn() -> NR,
NR: Into<String>;
}
```

Region

`Region` 은 회로의 특정 영역을 나타내며, 이 영역 내에서 셀에 값을 할당합니다:

```
pub struct Region<'r, F: Field> {
// 내부 구현
}

impl<'r, F: Field> Region<'r, F> {
// Advice 컬럼에 값 할당
pub fn assign_advice<V, VR, A, AR>(
&mut self,
annotation: A,
column: Column<Advice>,
offset: usize,
to: V,
) -> Result<AssignedCell<VR, F>, Error>
where
V: FnMut() -> Value<VR>,
VR: Into<Assigned<F>>,
A: Fn() -> AR,
AR: Into<String>;
// Fixed 컬럼에 값 할당
pub fn assign_fixed<V, VR, A, AR>(
```

```
&mut self,
annotation: A,
column: Column<Fixed>,
offset: usize,
to: V,
) -> Result<(), Error>
where
V: FnMut() -> Value<VR>,
VR: Into<Assigned<F>>,
A: Fn() -> AR,
AR: Into<String>;
}
```

AssignedCell

`assign_advice` 는 `AssignedCell` 을 반환하며, 이는 할당된 셀의 참조입니다:

```
pub struct AssignedCell<V, F: Field> {
cell: Cell,
value: Value<V>,
_marker: PhantomData<F>,
}
```

- **cell**: 셀의 위치 정보 (컬럼, 행)
- **value**: 할당된 값

- 다른 제약 조건에서 이 셀을 참조할 수 있음

assign 메서드 패턴

기본 패턴

대부분의 회로는 다음과 같은 `assign` 메서드 패턴을 따릅니다:

```
impl<F: Field> MyCircuitConfig<F> {
pub fn assign(
&self,
layouter: &mut impl Layouter<F>,
witness_data: &WitnessData,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
// 1. Region 생성
layouter.assign_region(
|| "assign witness",
|mut region| {
// 2. 각 행에 대해 반복
for (offset, row_data) in witness_data.iter().enumerate() {
// 3. 각 컬럼에 값 할당
region.assign_advice(
|| "column_name",
self.column,
```

```
offset,
|| row_data.value(challenges),
)?;
}
Ok(())
},
)?;
Ok(())
}
}
```

State Circuit의 assign 메서드

```
// zkevm-circuits/src/state_circuit.rs
pub fn assign(
&self,
layouter: &mut impl Layouter<F>,
rows: &[Rw],
n_rows: usize,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
// RW 연산을 정렬
let rows = rows.iter().sorted_by_key(|row| {
```

```
(
row.tag() as u64,
row.id().unwrap_or_default(),
row.address().unwrap_or_default(),
row.field_tag().unwrap_or_default(),
row.storage_key().unwrap_or_default(),
row.rw_counter(),
)
});
// Region 할당
layouter.assign_region(
|| "state circuit",
|mut region| {
self.annotate_circuit(&mut region);
let mut offset = 0;
// 각 RW 행 할당
for row in rows {
self.assign_row(&mut region, offset, row, challenges)?;
offset += 1;
}
// 패딩 행 할당
for _ in offset..n_rows {
self.assign_padding_row(&mut region, offset, challenges)?;
```

```
offset += 1;
}
Ok(())
},
)?;
Ok(())
}
```

Copy Circuit의 assign 메서드

```
// zkevm-circuits/src/copy_circuit.rs
pub fn assign_copy_events(
&self,
layouter: &mut impl Layouter<F>,
copy_events: &[CopyEvent],
max_copy_rows: usize,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
let copy_rows_needed = copy_events
.iter()
.map(|c| c.bytes.len() * 2)
.sum::<usize>();
layouter.assign_region(
```

```
|| "assign copy events",
|mut region| {
let mut offset = 0;
// 각 복사 이벤트 할당
for copy_event in copy_events.iter() {
self.assign_copy_event(
&mut region,
offset,
copy_event,
challenges,
)?;
offset += copy_event.bytes.len() * 2;
}
// 패딩
for _ in offset..max_copy_rows {
self.assign_padding_row(&mut region, offset)?;
offset += 1;
}
Ok(())
},
)?;
Ok(())
```

```
}
```

CellManager를 통한 셀 할당

CellManager의 역할

`CellManager` 는 회로의 셀 공간을 효율적으로 관리하는 유틸리티입니다. 두 가지 구현이 있습니다:

1. **EVM Circuit의 CellManager** (`zkevm-circuits/src/util/cell_manager.rs`): 수평 정렬 방식
2. **Keccak Circuit의 CellManager** (`zkevm-circuits/src/keccak_circuit/cell_manager.rs`): 수직 정렬 방식

Keccak Circuit의 CellManager

Keccak Circuit은 자체 CellManager를 사용합니다:

```
// zkevm-circuits/src/keccak_circuit/cell_manager.rs

pub struct CellManager<F> {

height: usize,

columns: Vec<CellColumn<F>>,

rows: Vec<usize>,

num_unused_cells: usize,

}



impl<F: Field> CellManager<F> {

pub(crate) fn new(height: usize) -> Self {

Self {
```

```
height,
columns: Vec::new(),
rows: vec![0; height],
num_unused_cells: 0,
}
}
// 셀 쿼리 (제약 조건 생성 시)
pub(crate) fn query_cell(&mut self, meta: &mut ConstraintSystem<F>) -> Cell<F>
let (row_idx, column_idx) = self.get_position();
self.query_cell_at_pos(meta, row_idx as i32, column_idx)
}
// 특정 행에서 셀 쿼리
pub(crate) fn query_cell_at_row(
&mut self,
meta: &mut ConstraintSystem<F>,
row_idx: i32,
) -> Cell<F> {
let column_idx = self.rows[row_idx as usize];
self.rows[row_idx as usize] += 1;
self.query_cell_at_pos(meta, row_idx, column_idx)
}
// 값 할당용 셀 쿼리
pub(crate) fn query_cell_value(&mut self) -> Cell<F> {
```

```
let (row_idx, column_idx) = self.get_position();
self.query_cell_value_at_pos(row_idx  as i32, column_idx)
}
// Region 시작 (모든 행을 같은 컬럼에서 시작)
pub(crate) fn  start_region(&mut  self) -> usize {
let  width = self.get_width();
for  row  in  self.rows.iter_mut() {
self.num_unused_cells += width - *row;
*row = width;
}
width
}
// 현재 너비 반환
pub(crate) fn  get_width(&self) -> usize {
self.rows.iter().cloned().max().unwrap()
}
// 최적 위치 찾기 (가장 적게 사용된 행)
fn  get_position(&mut  self) -> (usize, usize) {
let  mut  best_row_idx = 0usize;
let  mut  best_row_pos = 100000usize;
for (row_idx, row) in  self.rows.iter().enumerate() {
if *row < best_row_pos {
best_row_pos = *row;
```

```
best_row_idx = row_idx;
}
}
self.rows[best_row_idx] += 1;
(best_row_idx, best_row_pos)
}
}
```

Cell 구조체

```
pub(crate) struct Cell<F> {
pub(crate) expression: Expression<F>,
pub(crate) column_expression: Expression<F>,
pub(crate) column: Option<Column<Advice>>,
pub(crate) column_idx: usize,
pub(crate) rotation: i32,
}

impl<F: Field> Cell<F> {
// 값 할당
pub(crate) fn  assign(&self, region: &mut KeccakRegion<F>, offset: i32, value: F
region.assign(self.column_idx, (offset + self.rotation) as usize, value);
}
```

```
// Value<F> 할당
pub(crate) fn  assign_value(&self, region: &mut KeccakRegion<F>, offset: i32, va
let  value_f = extract_field(value);
region.assign(self.column_idx, (offset + self.rotation) as usize, value_f);
}
}
```

CellManager 사용 예시

```
// 제약 조건 생성 시
let  mut  cell_manager = CellManager::new(MAX_STEP_HEIGHT);

// 셀 쿼리
let  cell1 = cell_manager.query_cell(meta);
let  cell2 = cell_manager.query_cell_at_row(meta, 5);

// 제약 조건 추가
meta.create_gate("my constraint", |meta| {
vec![cell1.expr() - cell2.expr()]
});

// 할당 시
```

```
let value_cell = cell_manager.query_cell_value();

value_cell.assign(region, offset, value);
```

CellManager의 최적화

1. **수직 정렬**: 각 행에서 가장 적게 사용된 위치에 셀 할당
2. **Region 정렬**: `start_region()` 으로 모든 행을 같은 컬럼에서 시작
3. **미사용 셀 추적**: `num_unused_cells` 로 공간 효율성 모니터링

Region 관리

Region 생성

Region은 `layouter.assign_region()` 을 통해 생성됩니다:

```
layouter.assign_region(
|| "region name", // 디버깅용 이름
|mut region| {
// Region 내에서 셀 할당
Ok(())
},
)?;
```

여러 Region 사용

회로는 여러 Region을 사용할 수 있습니다:

```
// Region 1: 메인 데이터
layouter.assign_region(
|| "main data",
|mut region| {
// 메인 데이터 할당
Ok(())
},
)?;

// Region 2: 패딩
layouter.assign_region(
|| "padding",
|mut region| {
// 패딩 데이터 할당
Ok(())
},
)?;
```

Region 어노테이션

디버깅을 위해 Region에 어노테이션을 추가할 수 있습니다:

```
fn annotate_circuit(&self, region: &mut Region<F>) {
region.name_column(|| "rw_counter", self.rw_counter);
```

```
region.name_column(|| "is_write", self.is_write);

region.name_column(|| "tag", self.tag);

// ... 기타 컬럼

}
```

Offset 관리

Region 내에서 offset은 행 번호를 나타냅니다:

```
let mut offset = 0;


for row_data in witness_data {

region.assign_advice(

|| "value",

self.value_column,

offset,

|| Value::known(row_data.value),

)?;

offset += 1;

}
```

회로별 할당 예시

EVM Circuit 할당

EVM Circuit은 ExecutionGadget을 통해 각 실행 단계를 할당합니다:

```
// zkevm-circuits/src/evm_circuit/execution.rs

pub fn assign_exec_step(

&self,

region: &mut Region<F>,

offset: usize,

block: &Block<F>,

transaction: &Transaction,

call: &Call,

step: &ExecStep,

) -> Result<(), Error> {

// 1. Step 상태 할당

self.step.assign_exec_step(region, offset, block, transaction, call, step)?;

// 2. ExecutionGadget 선택 및 할당

match step.execution_state {

ExecutionState::ADD_SUB => {

self.add_sub_gadget.assign(region, offset, step)?;

}

ExecutionState::MUL_DIV_MOD => {

self.mul_div_mod_gadget.assign(region, offset, step)?;

}

// ... 기타 상태
```

```
_ => {}
}
Ok(())
}
```

Bytecode Circuit 할당

```
// zkevm-circuits/src/bytecode_circuit.rs
pub fn assign(
&self,
layouter: &mut impl Layouter<F>,
bytecodes: &[Bytecode],
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
layouter.assign_region(
|| "bytecode circuit",
|mut region| {
let mut offset = 0;
// 각 바이트코드 할당
for bytecode in bytecodes {
// 헤더 행
self.assign_header_row(&mut region, offset, bytecode, challenges)?;
offset += 1;
```

```
// 바이트 행들
for (idx, &byte) in  bytecode.bytes.iter().enumerate() {
self.assign_byte_row(
&mut  region,
offset,
bytecode,
idx,
byte,
challenges,
)?;
offset += 1;
}
}
// 패딩
for  _  in  offset..self.max_rows {
self.assign_padding_row(&mut  region, offset)?;
offset += 1;
}
Ok(())
},
)?;
Ok(())
```

```
}
```

Tx Circuit 할당

```
// zkevm-circuits/src/tx_circuit.rs
pub fn assign(
&self,
layouter: &mut impl Layouter<F>,
txs: &[Transaction],
max_txs: usize,
max_calldata: usize,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
// Tx Table 할당
layouter.assign_region(
|| "tx table",
|mut region| {
let mut offset = 0;
for tx in txs {
// 고정 필드 할당
for row in tx.table_assignments_fixed(challenges) {
self.assign_tx_row(&mut region, offset, &row)?;
offset += 1;
```

```
}
// 동적 필드 (CallData) 할당
for row in tx.table_assignments_dyn(challenges) {
self.assign_tx_row(&mut region, offset, &row)?;
offset += 1;
}
}
Ok(())
},
)?;
// RLP Circuit 할당
layouter.assign_region(
|| "rlp circuit",
|mut region| {
let mut offset = 0;
for tx in txs {
let rlp_witness = tx.gen_rlp_witness(true, challenges);
for row in rlp_witness {
self.assign_rlp_row(&mut region, offset, &row)?;
offset += 1;
}
}
Ok(())
```

```
},
)?;
Ok(())
}
```

Challenge 값 사용

Challenge란?

Challenge는 증명 생성 과정에서 검증자가 제공하는 무작위 값입니다. RLC(Random Linear Combination) 등에 사용됩니다.

Challenges 구조체

```
pub struct Challenges<T = Challenge> {
evm_word_challenge: T,
keccak_input_challenge: T,
lookup_input_challenge: T,
}

impl<T> Challenges<T> {
pub fn evm_word(&self) -> T {
self.evm_word_challenge
}
pub fn keccak_input(&self) -> T {
```

```
self.keccak_input_challenge
}
pub fn lookup_input(&self) -> T {
self.lookup_input_challenge
}
}
```

Challenge 값 적용

```
// RLC 계산
pub fn rlc_be_bytes<F: Field>(
bytes: &[u8],
challenge: Value<F>,
) -> Value<F> {
challenge.map(|r| {
bytes.iter().fold(F::zero(), |acc, &byte| {
acc * r + F::from(byte as u64)
})
})
}

// 할당 시 사용
region.assign_advice(
```

```
|| "value_rlc",
self.value_rlc_column,
offset,
|| rlc_be_bytes(&value_bytes, challenges.evm_word()),
)?;
```

Phase별 Challenge

Halo2는 여러 Phase를 지원하며, 각 Phase에서 다른 Challenge를 사용할 수 있습니다:

```
// FirstPhase: Challenge 없이 할당 가능
let column_first = meta.advice_column_in(FirstPhase);

// SecondPhase: FirstPhase의 Challenge 사용 가능
let column_second = meta.advice_column_in(SecondPhase);
```

최적화 기법

1. 배치 할당

여러 값을 한 번에 할당하여 성능 향상:

```
// 비효율적
for value in values {
region.assign_advice(|| "value", column, offset, || value)?;
offset += 1;
```

```
}

// 효율적
let assignments: Vec<_> = values
.iter()
.enumerate()
.map(|(i, value)| (offset + i, value))
.collect();

for (offset, value) in assignments {
region.assign_advice(|| "value", column, offset, || value)?;
}
```

2. 캐싱

반복적으로 사용되는 값을 캐싱:

```
// RLC 값 캐싱
let mut rlc_cache = HashMap::new();

for data in witness_data {
let rlc = rlc_cache.entry(data.key).or_insert_with(|| {
rlc_be_bytes(&data.bytes, challenges.evm_word())
});
```

```
region.assign_advice(|| "rlc", column, offset, || *rlc)?;
}
```

3. 조건부 할당

불필요한 할당 건너뛰기:

```
if  row.is_valid() {
region.assign_advice(
|| "value",
column,
offset,
|| Value::known(row.value),
)?;
} else {
// 패딩 행은 건너뛰거나 기본값 할당
region.assign_advice(
|| "padding",
column,
offset,
|| Value::known(F::zero()),
)?;
}
```

4. 병렬 처리

독립적인 Region은 병렬로 할당 가능:

```
use rayon::prelude::*;


// 여러 Region을 병렬로 처리
let regions: Vec<_> = witness_data
.par_iter()
.map(|data| {
// 각 데이터에 대한 할당 준비
prepare_assignment(data)
})
.collect();


// 순차적으로 할당
for region_data in regions {
layouter.assign_region(
|| "parallel region",
|mut region| {
assign_region_data(&mut region, &region_data)
},
)?;
```

```
}
```

할당 프로세스의 검증

제약 조건 검증

할당 후 Halo2는 자동으로 제약 조건을 검증합니다:

```
// 제약 조건 정의 (configure 시)
meta.create_gate("value consistency", |meta| {
let  value_cur = meta.query_advice(value_column, Rotation::cur());
let  value_next = meta.query_advice(value_column, Rotation::next());
vec![value_cur - value_next] // value는 일정해야 함
});


// 할당 시 이 제약 조건이 자동으로 검증됨
region.assign_advice(|| "value", value_column, 0, || Value::known(F::from(5)))?;
region.assign_advice(|| "value", value_column, 1, || Value::known(F::from(5)))?;
// region.assign_advice(|| "value", value_column, 1, || Value::known(F::from(6))
```

룩업 검증

룩업 테이블도 자동으로 검증됩니다:

```
// 룩업 정의 (configure 시)
```

```
meta.lookup("bytecode lookup", |meta| {
let  code_hash = meta.query_advice(code_hash_column, Rotation::cur());
let  byte_index = meta.query_advice(byte_index_column, Rotation::cur());
let  byte_value = meta.query_advice(byte_value_column, Rotation::cur());
vec![
(code_hash, bytecode_table.code_hash),
(byte_index, bytecode_table.index),
(byte_value, bytecode_table.value),
]
});


// 할당 시 룩업이 자동으로 검증됨
```

요약

할당 프로세스는 Witness 데이터를 회로에 실제로 배치하는 핵심 단계입니다:

1. **Layouter API**: Halo2의 `Layouter` 를 통해 Region 생성 및 관리
2. **assign 메서드**: 각 회로는 자신의 `assign` 메서드로 witness 할당
3. **CellManager**: 효율적인 셀 공간 관리 (수직/수평 정렬)
4. **Region 관리**: 여러 Region으로 회로 구조화, offset으로 행 관리
5. **Challenge 적용**: RLC 등에 사용되는 무작위 값 적용
6. **최적화**: 배치 할당, 캐싱, 조건부 할당, 병렬 처리

할당 프로세스는 `block_convert` 로 생성된 Block Witness를 받아, 각 회로의 제약 조건을 만족하도록 셀에 값을 배치하며, Halo2가 자동으로 제약 조건과 룩업을 검증합니다.

참고 자료

관련 코드:

- `zkevm-circuits/src/state_circuit.rs` : State Circuit 할당
- `zkevm-circuits/src/copy_circuit.rs` : Copy Circuit 할당
- `zkevm-circuits/src/bytecode_circuit.rs` : Bytecode Circuit 할당
- `zkevm-circuits/src/tx_circuit.rs` : Tx Circuit 할당
- `zkevm-circuits/src/util/cell_manager.rs` : EVM CellManager
- `zkevm-circuits/src/keccak_circuit/cell_manager.rs` : Keccak CellManager

관련 문서:

- Witness 개요
- Block Witness
- EVM Circuit
- State Circuit